CO3-AT1-Assignment

View Only

← Back

QUESTIONS

Q1 ✓
Banking Transaction System – Exception Handling
10 marks

Q2 ✓
Hospital Patient Registration – User-Defined Exception
10 marks

Q3 ✓
E-Commerce Order Processing – Built-in Exceptions
10 marks

Q4 ✓
Online Food Delivery – Multiple Threads and Execution Order
10 marks

Q5 ✓
Manufacturing Plant – Producer-Consumer Problem

Q4 Online Food Delivery – Multiple Threads and Execution Order

10 marks

An online food-delivery application uses three threads: Order Processing, Payment Processing, and Delivery Assignment.

Develop a Java program in which these activities are represented by separate threads. Analyze the problem if all three threads are started independently.

Modify the program so that:

- Order processing completes before payment processing.
- Payment processing completes before delivery assignment.
- The program uses appropriate Java thread-management techniques.
- The output clearly shows the required execution order.

Explain the difference between simply starting all threads and explicitly controlling their execution order.

Sample Test Cases:

Input / Scenario Expected Result Analysis Focus

Order=Pizza; Payment=UPI; Delivery=Agent A Order → Payment → Delivery Normal execution

10 marks

5/5 answered

Order=Burger; Payment=Card; Delivery=Agent B Order → Payment → Delivery Concurrency / design

Payment=Failed Delivery assignment should not proceed Exception / boundary

Your Code

```
1 import java.util.Scanner;
2 class Main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         String order=sc.nextLine();
6         String payment=sc.nextLine();
7         String delivery=sc.nextLine();
8         OrderThread t1=new OrderThread(order);
9         PaymentThread t2=new PaymentThread(payment)
10        DeliveryThread t3=new DeliveryThread(delive
11        try{
12            t1.start();
13            t1.join();
14            t2.start();
15            t2.join();
16            if(!payment.equalsIgnoreCase("Failed"))
17                t3.start();
18                t3.join();
19            }else{
20                System.out.println("Delivery assign
21            }
22        }catch(InterruptedException e){
23            System.out.println("Thread interrupted.
24        }
25        sc.close();
26    }
27 }
```

Input

```
Pizza
UPI
Agent A
```

Output

```
Order Processing:
Pizza
Order processing
completed.
Payment Processing:
UPI
```

```
class OrderThread extends Thread{
    private String order;
    OrderThread(String order){
        this.order=order;
    }
    public void run(){
        System.out.println("Order Processing: "+ord
        System.out.println("Order processing comple
    }
    public static void main(String[] args){
        Main.main(args);
    }
}
class PaymentThread extends Thread{
    private String payment;
    PaymentThread(String payment){
        this.payment=payment;
    }
    public void run(){
        System.out.println("Payment Processing: "+p
        if(payment.equalsIgnoreCase("Failed")){
            System.out.println("Payment processing
        }
    }
    public static void main(String[] args){
        Main.main(args);
    }
}
class DeliveryThread extends Thread{
    private String delivery;
    DeliveryThread(String delivery){
        this.delivery=delivery;
    }
```

```
61      public void run(){
62          System.out.println("Delivery Assignment: "+
63          System.out.println("Delivery assignment com
64      }
65      public static void main(String[] args){
66          Main.main(args);
67      }
68  }
```

Submitted